Cloud Premigration Advisor Tool (CPA...

KB139585

Copyright © 2025, Oracle. All rights reserved. Oracle Confidential.

Applies To

All Users Autonomous Database Dedicated - Version NA and later
Autonomous Database Serverless - Version N/A and later

Summary

The Cloud Premigration Advisor Tool (CPAT) helps you evaluate an existing Oracle database for compatibility with Autonomous Database (ADB) and other Oracle database offerings. Using CPAT before migrating to ADB makes assessing your source database easier and faster! CPAT removes much of the legwork of identifying potential user actions, prioritizing their importance, and suggesting resolutions. Oracle recommends using CPAT to accelerate your migration to Oracle Cloud.

- Cloud Premigration Advisor Tool (CPAT)
- Downloading and Extracting CPAT from the Zip File
- Supported Database Versions
  - Suggestions When Running on Oracle 10g
- Recent Changes to CPAT
- Basic Usage Notes and Requirements
  - Running CPAT
  - Required and Optional Environment Variables
  - Required Command Line Switches
  - Required Privileges

- Best Practices using Optional Command Line Switches
  - Fetch Target Instance Properties for Use in Source Instance Analysis
  - Limit the Analysis Done by CPAT Appropriately
  - Generate the JSON Report Even if You Do Not Need It and Always Save the Log File
  - Command Options that Help with Generating "What If" Scenarios
  - Limiting the Amount of Data in Reports
  - Forcing Reports to Ignore Locale Settings and be in English
- Invoking CPAT Using a Parameter File
  - Parameter File without Namespaces for Single Database
  - Parameter File with Multiple Namespaces / Databases (Interactive Mode)
  - Setting Up Keystore to Store Passwords (Interactive Mode Only)
  - No Console Mode (Multiple Namespaces Without Interactive Mode)
- Additional Usage Notes
  - Target Cloud Values
    - Automatic Detection of Target Cloud
  - Migration Methods
    - XTTS Support
  - Connection Strings
  - Analyzing Multiple PDBs in a CDB
  - Generating "Fixups" for Checks That Do Not Pass
  - Gathering Extra Information on Source Database
  - Scope of CPAT Checks
  - Full vs Schema "Mode"
    - FULL mode
    - SCHEMA mode

        - Controlling CPAT's Mode
        - Additional Mode Notes
    - HTML Report When Offline
    - Interpreting CPAT Report Data
        - Relevant Objects Included for Checks in CPAT Reports
        - Why are Checks Sometimes Marked as "skipped"
    - Advanced Filtering for Reported Objects
- Checking for an Updated Version of CPAT
- CPAT Integration with Other Oracle Products
- Updating CPAT in a Zero Downtime Migration Home
- Using CPAT with Other Tooling or Scripted Processes
    - CPAT Shell Script Exit Codes
- Security Notes
- Summary

Solution

The Cloud Premigration Advisor Tool can perform analysis of both the source and the target database and provide information about the suitability of migrating the source database to an Oracle Cloud offering.

This document describes what CPAT does, where to get it, and how to use it.

Cloud Premigration Advisor Tool (CPAT)

CPAT is a Java based tool that connects to an Oracle database in order to perform a series of checks. Each check is designed to evaluate a particular set of objects or conditions to ensure a successful migration to an Oracle Cloud offering.

Once the checks are performed CPAT will generate a report indicating what was found. Reports contain both summary information and details for each check including the check

"result" (e.g. **Passing**, **Review Suggested**, **Review Required**, **Action Required**) and what "relevant data" was found in the source database. CPAT can generate reports in HTML, TEXT, and JSON format.

Downloading and Extracting CPAT from the Zip File

Download the most recent version of CPAT from here.

- CPAT 26.2.0 *(February 27th, 2026)*

Note that the CPAT application itself is not tied to a particular database version. Therefore there is a single download for CPAT and that one download can be used for all supported versions of the Oracle database (11.2.0.4 and higher)

Once downloaded, use a standard unzip utility to unzip the CPAT kit.

Supported Database Versions

CPAT was designed to work with Oracle 11.2.0.4 Enterprise Edition and newer versions. CPAT may run successfully on some earlier versions (10g) and other editions (e.g. Standard) but running CPAT on these other versions and editions is not officially supported by Oracle. The results are on a 'best effort' basis. Some validation checks may not work properly, and extra care must be assumed by the user when interpreting results.

Suggestions When Running on Oracle 10g

CPAT does not officially support Oracle database versions lower than 11.2.0.4, but it has been known to run successfully with Oracle 10g. There is a known issue with using the Oracle 10g JDBC libraires however. When using native 10g libraries, instead of the libraries provided in the CPAT kit, CPAT fails to run. So, to make sure to use the JDBC libraries from the CPAT kit, the following steps can be followed:

- Don't define ORACLE_HOME.
- Use the thin driver when connecting to the Oracle 10g database.

If any other errors are encountered, there is no other recommended actions. There is no official support for Oracle 10g.

Recent Changes to CPAT

A list of recent changes to CPAT can be found here.

Basic Usage Notes and Requirements

Running CPAT

The distribution kit contains **premigration.sh** for running CPAT on Unix platforms and **premigration.cmd** for running CPAT on Windows platforms.

CPAT can be run from any host that has network access to the database being analyzed.

Required and Optional Environment Variables

CPAT, as a Java based tool, requires that an appropriate Java Runtime Environment (JRE) be present on the machine where CPAT is run. The minimum JRE version required for CPAT is **Java 8**.

CPAT will look for a JRE using **JAVA_HOME** and/or **ORACLE_HOME**.

If using a "thick", Oracle Call Interface (OCI) based JDBC connect string then CPAT currently expects the following environment variables to be set: **ORACLE_SID, ORACLE_HOME,** and **LD_LIBRARY_PATH**.

More details on connect strings and associated environment variables can be found in the *Additional Usage Notes* section titled Connection Strings.

Note: The recommended way to set the **ORACLE_SID, ORACLE_HOME,** and **LD_LIBRARY_PATH** variables on Unix is by using the "oraenv" script that comes with Oracle Database.

If the version of Java in **ORACLE_HOME** is Java 7 (or lower) then setting **JAVA_HOME** must bet set to a **Java 8** (or higher) JRE.

If you want to use CPAT without defining **ORACLE_HOME** and don't need to use a "thick" (OCI based) JDBC connection string then as long as **JAVA_HOME** is set to a **Java 8** (or higher) JRE then CPAT's Java requirements will be met.

Required Command Line Switches

When using CPAT to connect to a database for source analysis:

- Only "**--targetcloud <adb-type>**" and "**--connectstring <jdbc-connect-string>**"are typically required assuming the connect string specifies a wallet with a username and password.
- If using Operating System authentication or the SYS user then "**--sysdba**" would also be required.
    - "**--sysdba**" would also be required when connecting as a user who has been granted SYSDBA but not the other privileges required by CPAT (see "Required Privileges")
- If not using a wallet or Operating System authentication then "**--username**" is required.

For example:

```
./premigration.sh --connectstring jdbc:oracle:oci:@
--targetcloud ATPD --sysdba
```

Use the "**--help**" switch for the full list of options and their descriptions. For example:

```
./premigration.sh --help
```

For easy reference a copy of the output from running CPAT with "--help" is attached to this note.

Required Privileges

When employing the "**--username**" switch with CPAT the supplied Oracle user name must have the SELECT ANY DICTIONARY privilege and have been granted SELECT on SYSTEM.DUM$COLUMNS and SYSTEM.DUM$DATABASE.

Access to the DUM$ tables is needed only if the source and target character sets indicate that Oracle Database Migration Assistant for Unicode (DMU) is required.

When using an ADB serverless instance as the source for CPAT analysis, the user running CPAT should be granted the SELECT_CATALOG_ROLE role as described in the Autonomous Database Serverless documentation.

When using the XTTS migration method, the user running CPAT will also need EXECUTE privileges on SYS.DBMS_TTS.

Note: The installation and running of CPAT does not modify the Oracle Database. CPAT does not create any users, any packages, or require granting any roles or privileges. CPAT treats the database as **READ ONLY**.

Best Practices using Optional Command Line Switches

Fetch Target Instance Properties for Use in Source Instance Analysis

Note: Using CPAT with a properties file generated from the target instance is **strongly** recommended.

Certain properties of the target database instance are

required in order to perform the most complete analysis possible of the source database instance. The "**--gettargetprops**" switch is intended to be used with the other connection related switches to perform this function.

For example:

```
./premigration.sh --gettargetprops -username ADMIN
--connectstring 'jdbc:oracle:thin:@<service-name>?
TNS_ADMIN=<path-to-wallet>'
```

The command generates a premigration_advisor_analysis.properties file that can be used when analyzing a source instance.

If necessary the properties file should be copied to the host where analysis of the source database will be done and supplied to CPAT using the "**--analysisprops**" switch.

For example:

```
./premigration.sh --connectstring jdbc:oracle:oci:@
--targetcloud ATPD --sysdba --analysisprops
premigration_advisor_analysis.properties
```

If you know that you (or ZDM or DMS) will be mapping (or pre-creating) all needed tablespaces then append **MigrationMethodProp.ALL_METHODS.TABLESPACE_MAPPING=ALL** to the properties file passed to CPAT. This will cause CPAT to PASS most (if not all) of its tablespace related checks. However, customers should keep in mind that there could still migration issues related to quotas with tablespace mapping.

Limit the Analysis Done by CPAT Appropriately

You can use "**--schemas <space-separated-list-of-schemas>**" to restrict what schemas CPAT will examine when performing its analysis. If the "--schemas" switch is not used, CPAT will analyze all schemas in the source instance (excluding schemas maintained by Oracle) which may result in problems being found in schemas that you do not intend to migrate. See the "Scope" section of this MOS Note to get a better idea of if you should also use the "**--full**" switch. This approach might be particularly useful if the source instance is used to host multiple applications, each of which you may want to migrate to different ADB instances. If you want to

analyze all schemas that are not maintained by Oracle, but still run in schema mode, you can use the **--schemas** parameter with the reserved word "**ALL**", so that you do not have to list every schema manually in the command line.

Generate the JSON Report Even if You Do Not Need It and Always Save the Log File

The "--reportformat" switch accepts one or more report formats separated by spaces. Allowed values for the --reportformat switch are "TEXT", "HTML", and "JSON". If you encounter an issue when using CPAT, it is very important to include all possible information when contacting support. **This includes the .log file and all generated reports**.

An easy way to gather all files generated by CPAT is to use the "--zip" command line switch. When this switch is used all generated files will be combined into a zip file. All generated files will remain in place should you need to use or view them separately.

Command Options that Help with Generating "What If" Scenarios

When exploring multiple "what if" scenarios it's handy to use the **--outfileprefix** and **--outdir** switches to keep things straight and to keep reports from being overwritten.

For example:

```
./premigration.sh --outfileprefix ATPS_RUN_01 --
outdir /path/CPAT_output --reportformat HTML JSON
...
```

The "**--outdir**" switch accepts an absolute or relative folder path. Using this switch directs CPAT to where the log files, report files, and/or properties files should be created. If omitted the log file and other generated files will be created in the current folder which can lead to files being overwritten when multiple analyses are performed.

Limiting the Amount of Data in Reports

Certain CPAT checks may return upwards of thousands of objects. To make TEXT reports less overwhelming consider using "**--maxtextdatarows <number>**". That will limit the amount of objects to the number passed to the flag.

Using "**--maxrelevantobjects <N>**" will have the same affect but also limit the HTML and JSON content. However this is typically not necessary.

Forcing Reports to Ignore Locale Settings and be in English

CPAT will automatically try to translate reports depending on the locale settings of the machine in which it was run. However, there may be times when it makes sense to leave the report in English (such as when working in teams with members spread out globally). To force CPAT to ignore system locale settings and deliver the reports in English, the **-E** / **--enlang** flag can be used.

Invoking CPAT Using a Parameter File

Parameter File without Namespaces for Single Database

As an alternative to passing CPAT parameters via the command line, CPAT will also accept parameters via a configuration file.

To use this feature, run CPAT with the "**--parfile**" command line option and supply the name of the parameter file.

For example:

```
./premigration.sh --parfile cpatParams.cfg
```

Ensure the file contains all required parameters using the long command line options available in the CPAT command line tool (use "**./premigration.sh --help**" for a list of long command line options).

Parameter names must be followed by an equals sign and then the desired value for that parameter.

When a parameter accepts multiple values those must be comma separated in the parameter file.

Example configuration file:

```
connectstring=jdbc:oracle:thin:@example.com:1521/OR
CLCDB
pdbname=ORCLPDB1
username=SYSTEM
targetcloud=ATPS
outdir=./cpatOutput
reportformat=TEXT,JSON,HTML
```

For "boolean" command line options that are toggled on such as "**--genfixups**", use "**genfixups=true**" in the parameter file.

Parameter File with Multiple Namespaces / Databases (Interactive Mode)

A parameter file can also list parameters for several different databases by using namespaces to tag the different databases. The file must be set up the same way the single database parameter is set up, but the parameter names must be prefixed by a namespace alias followed by a period. For example, to use namespace "**db1**" to represent a database, the connect string parameter for that would look like "**db1.connectstring=**". Any alphanumerical string can be used to define a namespace. However, there are two reserved words which cannot be used, the word "**all**", and the word "**global**". The word global can be used to add parameters that will impact all defined databases in the configuration file. But, a global parameter will be overwritten if the same parameter is listed with a specific namespace.

Using namespaces in the configuration file will automatically kick off CPAT's new interactive mode. This new interactive mode can be used to list out all databasese set up, run CPAT for any of those databases, and look at the progress of any current runs. This interactive mode has its own list of

supported commands. Once in the interactive mode, typing "**help**" will display the list of available commands, how they should be used, and a description of what they do. To run CPAT for a specific database, command "**run <namespace>**" can be used. That will trigger a CPAT run for the database tagged by the namespace used. Similarly, "**run all**" can be used to run CPAT for all defined databases at the same time. Passwords will be prompted for the users with which database connection is going to happen according to the configuration file.

The more databases that are run at the same time, the harder time the system will have running CPAT

Example configuration file:

```
# Global parameters, which will apply to all
databases
global.targetcloud=ATPS
global.reportformat=html
global.full=true
global.migrationmethod=DATAPUMP
global.analysisprops=/var/cpat/propFiles/premigrati
on_advisor_analysis.properties
global.username=SYSTEM
global.gatherdetails=ALL

# Parameters specific to first database
db1.outdir=/var/cpat/premigration/db1
# The following line overwrites the global value
for gatherdetails for this database
db1.gatherdetails=NONE
db1.connectstring=jdbc:oracle:thin:@example.com:152
1/ORCLCDB1

# Parameters specific to second database
db2.outdir=/var/cpat/premigration/db2
db2.connectstring=jdbc:oracle:thin:@example.com:152
2/ORCLCDB2
...
```

Setting Up Keystore to Store Passwords (Interactive Mode Only)

When running in interactive mode, an extra "**keystore**" parameter can be added to the configuration file. This parameter takes in a location where a keystore or wallet can be stored. If set up correctly, CPAT will look for passwords in that wallet and will use a password automatically (without prompting) if the password for the correct user is set up. The

parameter must be set using the global namespace, as only one wallet is supported at this time. The line to add the keystore parameter would look like:

```
global.keystore=/var/cpat/wallet
```

CPAT provides an interactive mode, "**CPAT Password Loader**", to set up passwords in the wallet. This mode can be accessed by running the parameter file option with a new "**--loadpassword**" option:

```
./premigration.sh --parfile cpatParams.cfg --
loadpassword
```

Using the "**--loadpassword**" option when the configuration file specifies the keystore path but no wallet exists will result in CPAT creating a new wallet. It's a password protected wallet so it will prompt for a master password for the wallet. If the wallet is already created, then it will prompt for the wallet's password. This interactive mode, just like the parameter file interactive mode, has its own supported commands. Typing "**help**" in the command line will provide a list of available commands. Moreover, typing a command that needs extra parameters to work will provide examples on how to call the command. This interactive mode can be used to set passwords for different users in the different databases (namespaces) configured. The following is an example of how the password for users SYSTEM and DUMMY can be set up for database "**db1**", and the password for user DUMMY can be set up for database "**db2**". The system would prompt for the user's password and the validation of the password after each command:

```
add db1 -user SYSTEM
Enter your secret/Password:
Re-enter your secret/Password:
add db1 -user DUMMY
Enter your secret/Password:
Re-enter your secret/Password:
add db2 -user DUMMY
Enter your secret/Password:
Re-enter your secret/Password:
```

When setting up a password, CPAT makes sure that the correct password was provided by trying to log into the database with the provided user and password. This may take some time as it depends on the connectivity to the database. In cases where the user is completely sure of the password and/or there is no database connectivity, the "**-nocheck**" parameter can be passed into the "**add**" command.

Passwords may also be listed, or removed from the wallet. Also, changes to the wallet itself can be saved. When trying to save changes to the wallet, CPAT will prompt for the wallet to be configured as "**auto-login**". If YES is chosen, then the wallet won't prompt for a password unless adding or removing a user from it. This helps greatly in "no console" mode (discussed in the below section).

After password set up, going back to parameter file interactive mode will prompt for the global wallet password, but will not prompt for the password for a user if that user has been properly set up in the wallet, so it can save a considerable amount of time when running several databases at the same time.

No Console Mode (Multiple Namespaces Without Interactive Mode)

There is a way for CPAT to skip interactive mode and try to run CPAT for all configured databased in the configuration file at once. This can be done by using the "**--noconsole**" option when running in CPAT in parameter file mode:

```
./premigration.sh --parfile cpatParams.cfg --
noconsole
```

Running with the "**--noconsole**" flag depends on passwords being previously set up in a wallet (see the Setting Up Keystore to Store Passwords section above), as the only password this mode will prompt for is the wallet password (if it was not set up for "**auto-login**"). If it does not find the password for a particular user@database, then it will just skip running CPAT for that database. To prevent the wallet password from being prompted, the wallet can either be set up for "**auto-login**" in "**CPAT Password Loader**", or the password may be piped into the command, though this is not recommended for security reasons:

```
echo "SuperComplexPassword" | ./premigration.sh --
parfile cpatParams.cfg --noconsole
```

Additional Usage Notes

Target Cloud Values

The "**--targetcloud**" switch accepts values indicating what Oracle Cloud offering is the destination of the intended migration. The values ATPD, ADWD, ATPS, and ADWS represent each of the basic Autonomous Databases types.

Specifying "**--targetcloud DEFAULT**" tells CPAT that the destination is not Oracle Autonomous Database but rather an instance of the Oracle Database that does not employ a PDB_LOCKDOWN.

For more information on PDB_LOCKDOWN parameters see the PDB_LOCKDOWN parameter in the 19c Database Reference.

Automatic Detection of Target Cloud

Because CPAT fetches PDB_LOCKDOWN and CLOUD_SERVICE properties from the target instance it is possible to derive the target cloud value even if the "**--targetcloud**" switch is not used. This works for only a specific set of target instance property values:

Values for Target Cloud Derivation

Target PDB_LOCKDOWN	PAAS	ADW	OLTP	DWCS	<missing> or <other-profile>	<empty-string>
Target CLOUD_SERVICE	PAAS	PAAS	OLTP	DWCS	<missing> or <other-service>	<empty-string>
Default --targetcloud value	ATPD	ADWD	ATPS	ADWS	None; --targetcloud must be provided	DEFAULT

If you use "**--targetcloud**"with a properties file that supplies the above properties CPAT will detect any mismatch between the command line value and the properties file values. A mismatch will result in a warning being displayed but analysis will continue. Note that a mismatch could result in some checks returning unexpected data and requires additional scrutiny by end users.

Migration Methods

By default CPAT assumes that the migration to the cloud instance will be a logical migration done using standard Oracle Data Pump dump files (CPAT does not support physical migration checks at this time).

However, if the migration will be done using Oracle Data Pump over a DBLINK or if Oracle GoldenGate will be used in addition to Oracle Data Pump, users should explicitly use the "--**migrationmethod**" to indicate what methods will be used to perform their migration.

For example: specifying "**--migrationmethod DATAPUMP_DBLINK GOLDENGATE**" would result in CPAT checking for anything in the source database that can be problematic for Oracle Data Pump when used over a DBLINK or when Oracle GoldenGate is employed.

The reserved word **ALL** can be specified to lump all migration methods together (DATAPUMP DATAPUMP_DBLINK GOLDENGATE).

XTTS Support

CPAT can now support the physical migration method Cross-platform Transportable Tablespaces (XTTS). The specific migration type it currently supports is the XTTS migration using the M5 script, but we will continue adding functionality for other TTS-based migration types. XTTS is a migration method that changes the scope of certain checks, so it can only be used by itself, e.g. "**--migrationmethod XTTS**". If any other method is used along with it, it will default to using the other methods and XTTS will be explicitly removed from the list. XTTS will be ignored when using the reserved word ALL.

As the other migration methods take advantage of the --schemas or --excludeschemas flags to limit the amount of users to consider for migration, the XTTS migration method will infer the list of tablespaces to migrate from the **--tablespaces** or **--excludetablespaces** flags. These are analogous to their schema counterparts, and will work the same (a list of tablespaces separated by spaces). For the **--tablespaces** flag, only the tablespaces listed will be considered. For the **--excludetablespaces** flag, all tablespaces except those listed will be considered.

Note: To run CPAT with XTTS as a migration method, the user running CPAT will need EXECUTE privileges on SYS.DBMS_TTS additionally to all other required privileges.

Connection Strings

CPAT accepts standard Oracle JDBC format connection strings. This means that both the "thick" and "thin" Oracle JDBC driver can be used. The "thin" driver is often preferred due to less required configuration (only JAVA_HOME needs to be defined, not ORACLE_HOME). Also, CPAT ships with the necessary jars for "thin" driver use.

Examples:

Example JDBC connection strings

Connection Description	Connection String	Notes
Thin	jdbc:oracle:thin:@<host>:<port>:<sid>	
Thin with PDB Service	jdbc:oracle:thin:@<host>:<port>/<pdb-service-name>	
Thin with AWS RDS	jdbc:oracle:thin:@database-1.xxx.us-east-1.rds.amazonaws.com:<port>:<sid>	Consult the AWS RDS documentation for instructions on finding your database's endpoint and port details. See this blog for an example.
OS Authentication	jdbc:oracle:oci:@	Command line must also include "--sysdba"
OS Authentication with PDB	jdbc:oracle:oci:@	Command line must also include "--sysdba" and "--pdbname <pdb-name>"
All PDBs in a CDB	jdbc:oracle:thin:@<host>:<port>/<cdb-service-name>	

Wallet based with Java 8 JRE	jdbc:oracle:thin:@<service-name>?TNS_ADMIN=<path-to-wallet>	This example is derived from Oracle Autonomous documentation

When using connection strings such as "jdbc:oracle:thin:@<service-name>?TNS_ADMIN=<path-to-wallet>" JDBC requires that either,

1. an ojdbc.properties file be located the wallet-dir and contain the oracle.net.wallet_location property and value such as "oracle.net.wallet_location=(SOURCE=(METHOD=FILE)(METHOD_DATA=(DIRECTORY=${TNS_ADMIN})))", or,
2. the JAVA_TOOL_OPTIONS environment variable be set with the appropriate values such as "export JAVA_TOOL_OPTIONS='-Doracle.net.tns_admin=<path-to-wallet-dir> -Doracle.net.wallet_location=(SOURCE=(METHOD=FILE)(METHOD_DATA=(DIRECTORY=<path-to-wallet-dir>)))'"

Please see Document ID 1123423.1, "How To Use an External Password Store with JDBC/Thin" for more information.

When using Kerberos security with a connection one can specify the needed configuration by using an environment variable such as "export JAVA_TOOL_OPTIONS='-Djava.security.krb5.conf=/path/to/krb5.conf'"

Analyzing Multiple PDBs in a CDB

Using "**--pdbname**" is only required when the connection string is for CDB$ROOT and you do not wish to analyze all the PDBs in the CDB.

Supplying multiple PDB names with "**--pdbname**" will instruct CPAT to analyze just those named PDBs in the CDB.

Generating "Fixups" for Checks That Do Not Pass

Note: Use caution in handling these generated scripts as they may contain sensitive information. It is advised to perform a backup and consult the latest documentation before executing any scripts.

Some of the migration impediments that CPAT checks for can be addressed by running SQL statements on either the source or the target instance.

By using the '**--genfixups**' command line switch when performing analysis of your source instance CPAT will generate scripts for running these SQL statements.

Any scripts generated by CPAT will be created below the folder indicated by the "**--outdir**" command line switch passed to CPAT or the current folder if "**--outdir**" is not used.

Those scripts intended to be used on the source instance will be created in a subfolder called "source" and scripts intended to be used on the target instance will be created in a subfolder called "target".

Gathering Extra Information on Source Database

CPAT can now gather details on the source database, which may help get a better perspective of the size and composition of said database. These details can be fetched by using the "**--gatherdetails**" parameter.

This parameter accepts one or more options separated by spaces. The allowed options are as follows:

--gatherdetails Options

NONE	This option gathers no extra details. This is the default value, it's the same as not using the parameter
SCHEMA_OBJECTS	This is information on the objects in the database. You can find information such as a summary of all objects in the database, object and column type distribution, the largest 50 objects, table and index segments, secure files, etc.
OEE	This is information necessary for the OEE integration. It gathers extra information on tablespaces and

	datafiles, database options, database parameters, NLS parameters, and source instance information
ALL	Gathers all possible details

You can find the SCHEMA_OBJECTS information, in both the HTML and text reports, in a section called "**Source Database Schema Summary Information**". In the json report, you can fetch the same information from the "**srcSchemaSummary**" option.

You can aso find the OEE information, in both the HTML and text reports, in sections appropriately named "**Source Database Tablespace Information**", "**Source Database Options Information**", "**Source Database Parameter Information**", "**Source NLS Database Parameter Information**", and "**Source Instance Information**". In the json report, those sections correspond to options "**srcDatabaseTablespaceInfo**", "**srcDatabaseOptions**", "**srcDatabaseParameters**", "**srcNlsDatabaseParameters**", and "**srcInstanceState**".

While "**--gatherdetails**" can take more than one option, using NONE or ALL will overwrite any of the options used and will provide either no details, or all possible details. If you use both NONE and ALL, ALL will be picked over NONE.

Example use:

```
./premigration.sh -u CPAT_USER -r HTML ... --
gatherdetails SCHEMA_OBJECTS OEE
```

Scope of CPAT Checks

The scope of CPAT checks fall into one of four categories: SCHEMA, SCHEMA_ONLY, INSTANCE, and UNIVERSAL

CPAT has checks that are only applicable to data within schemas. For example, CPAT can check schemas for tables that use deprecated and desupported features such as Oracle Multimedia object types. Such checks are SCHEMA scope.

CPAT has checks that are only applicable to data within schemas, like above, but are not appropriate when CPAT is running in "FULL" mode. Such checks are SCHEMA_ONLY scope.

CPAT has checks that are only applicable to the database instance as a whole. For example, CPAT can check for certain

parameters and values in the V$PARAMETER table. Such checks are INSTANCE scope.

CPAT also has checks that should *always* be run. Such checks are UNIVERSAL in their scope.

Full vs Schema "Mode"

FULL mode

By default CPAT runs all applicable checks and analyzes data in **all** appropriate schemas. This is termed **FULL** mode. In full mode SCHEMA, INSTANCE, and UNIVERSAL scope checks are run.

SCHEMA mode

If the "--schemas" switch is used (and "--full" not used) then CPAT will run in **SCHEMA** mode. In schema mode SCHEMA and UNIVERSAL scope checks are run. INSTANCE scope checks are not run.

Controlling CPAT's Mode

The CPAT mode is controlled by the use of two options switches: the "**--schemas**" switch and "**--full**" switch.

When the "-**-schemas**" switch is **not** used then CPAT operates in FULL mode.

When the "**--schemas <schema-name-list>**" switch is used on the command line CPAT runs in SCHEMA mode unless the "**--full**" switch is also used, in which case CPAT operates in FULL mode.

Note that <schema-name-list> is a space separated list. For example: "premigration.sh --schemas PARDUS COMEDIT FACIEM MEAM --targetcloud ATPS --connectstring <jdbc-connect-string>"

Please use the "**--help**" command line switch to get details on the use of all command line switches and arguments.

Additional Mode Notes

It should be noted that even in FULL mode CPAT will by default *exclude* checking data in schemas known to be maintained by Oracle.

The use of the "**--excludeschemas**" switch does not change CPAT's default FULL mode.

Whichever mode is used with CPAT should also be used with Data Pump. Run CPAT in "full" mode if you intend to use Data Pump with "full" mode and run CPAT in "schema" mode if you intend to use Data Pump in "schema" mode.

HTML Report When Offline

The HTML report can now be shipped with all dependencies so they can work even when no internet connection is available. This has been made available as a new parameter for the --reportformat / -r flag, **HTML_OFFLINE**.

Example use:

```
./premigration.sh -u CPAT_USER -c ... --
reportformat HTML_OFFLINE
```

The dependencies folder, will be named **offline_html_dependencies**. This folder will always need to be placed in the same location the report is placed. This is what CPAT does initially. But if the report is moved manually, then the dependencies folder must also be moved along with the report.

Interpreting CPAT Report Data

Reports generated by CPAT contain both summary information and details for each check successfully performed.

Included in details for each check are the fields:

- **Description**: describes what the check is looking for or why the check is being performed
- **Failure Impact**: describes the consequences of a result other than **Passing**
- **Action**: describes what, if anything, should be done in order to address the situation should the check result not be **Passing**

Each check successfully run by CPAT will have a result of **Passing**, **Review Suggested**, **Review Required**, or **Action Required**. If a check fails to be run, or CPAT itself encounters an irrecoverable error, a result of **Failed** is returned.

The overall result of the CPAT report will be the most severe result of all checks performed. For example, if 30 checks **Passing** but one check has a **Review Required** result the overall result will be **Review Required**.

The current definitions of each of the CPAT check results are as follows:

CPAT Check Result Definitions

Passing	Indicates migration should succeed and applications should see no difference in behavior.
Review Suggested	Indicates migration should succeed and applications will likely see no functional difference but database administrators should evaluate each Check with this status to look for any unforeseen consequences.
Review Required	Indicates migration may succeed (at least in part) but not everything is expected to work exactly like before or some work post migration must be done to bring the target instance into alignment with the source database.
Action Required	Indicates something that would likely cause the migration to be unsuccessful. Checks with this result typically need to be resolved before attempting migration.
Failed	The Cloud Premigration Advisor was unable to complete its analysis. Please contact Oracle Support Services.

Note: A CPAT result of **Action Required** does not necessarily mean that, for instance, Oracle Data Pump import will terminate prematurely while importing the data. It means that there will likely be errors during import which can indicate not all data has been migrated. It is imperative that an administrator familiar with both the database and the applications layered on the database examine the results of any checks that are not **Passing** but especially **Review Required** and **Action Required** results.

Relevant Objects Included for Checks in CPAT Reports

Relevant objects are any objects pertinent to the check being performed. Relevant objects may literally be database objects such as tables. Relevant objects may be database parameters

or other settings. While frequently the case the absence of relevant objects does not necessarily mean a check will pass. Nor does the presence of relevant objects mean that a check will not pass.

Most checks that don't pass and also report "0 relevant objects" do so because CPAT was not provided an "analysis properties file" generated from the target instance.

Why are Checks sometimes marked as "skipped"

Checks that are "skipped" would normally have been performed for the given CPAT parameters (--targetcloud, --migrationmethod, etc) but ultimately are not run.

The two cases are:

1. The Check *should* be run but *cannot possibly* be run due to the current contents or configuration of the source database. The CheckResult will be **Review Suggested** or more severe.
2. There is no need to run the check due to the current contents or configuration of the source database. The CheckResult will be **Passing**.

Advanced Filtering for Reported Objects

Historically, the granularity of CPAT's analysis and reporting was limited to the schema level (for example, by using **--schemas** and **--excludeschemas**). CPAT now supports additional filtering at the object level so that specific owners, object types, or named objects can be excluded from CPAT reports.

The following new command line switches can be used to filter objects from the report:

- **-O / --reportexcludeowners**
  Excludes all objects owned by schemas matching the supplied patterns. This parameter accepts a comma-separated list of schema patterns. Wildcards are supported.
  Example:

```
--reportexcludeowners LEGACY_*,TEST_*
```

- **-T / --reportexcludetypes**
  Excludes objects of the specified object types. This parameter accepts a comma-separated list of object types.
  Example:

  ```
  --reportexcludetypes TABLE,PROCEDURE
  ```

- **-X / --reportexcludeobjects**
  Excludes objects matching the supplied fully-qualified object name patterns. This parameter accepts a comma-separated list of object names, and the full name must be provided in the form **<schema>.<object_name>**. Wildcards are supported for the schema name and/or object name.
  Examples:

  ```
  --reportexcludeobjects
  SCHEMA.TEMP_*,*._BAK__*
  ```

These parameters provide finer control over what appears in CPAT's output, which can be useful when generating reports intended for specific application teams or when excluding objects that are known to be out of scope for migration. All filtering parameters can be used in tandem as needed.

Any objects filtered by any of the rules used will be removed from the list of relevant objects, which is the list used to compute the actual result. However, the json report will hold a new section, **postFilteredObjectsData**, where all of the filtered objects will be displayed for traceability/auditing purposes

Additionally, **-Y / --reportfilterfile** can be used to pass in the path to a JSON or YAML formatted file with the desired post-execution filter rules.

Example:

```
{
    "_comment": "CPAT Post-Execution Filter
Configuration File",
    "_documentation": {
```

```
        "purpose": "This file defines rules to
filter objects from CPAT report results after
analysis is                                 complete",
        "usage": "./premigration.sh --connectstring
'...' --username SYSTEM --reportfilterfile
./sample-filter-                     config.json",
        "rule_evaluation": "Rules are evaluated in
order. First matching rule determines the action
(INCLUDE/                                  EXCLUDE)",
        "pattern_syntax": "Patterns use Java regex
syntax. Common patterns: ^prefix.* (starts with),
.*suffix$                                 (ends
with), .* (any)"
    },

    "settings": {
        "_comment": "Global settings for filter
behavior",
        "empty_check_behavior": "keep_pass",
        "_empty_check_behavior_options": {
            "keep_pass": "When all objects in a
check are filtered out, set check result to PASS",
            "keep_original": "When all objects are
filtered, keep the original check result (BLOCKER,
WARNING,                                   etc.)"
        }
    },

    "rules": [
        {
            "_comment": "EXAMPLE 1: Exclude objects
by name pattern (regex)",
            "name": "exclude_temp_tables",
            "description": "Exclude temporary
tables that start with TEMP_ prefix",
            "action": "EXCLUDE",
            "match": {
                "_comment": "match criteria - all
conditions must be true (AND logic)",
                "object_name": "^TEMP_.*",
                "object_type": "TABLE"
            }
        },

        {
            "_comment": "EXAMPLE 2: Exclude objects
by name suffix pattern",
            "name": "exclude_backup_objects",
            "description": "Exclude backup/archive
objects ending with _BAK, _BACKUP, _OLD, or
_ARCHIVE",
            "action": "EXCLUDE",
            "match": {
                "object_name":
".*_(BAK|BACKUP|OLD|ARCHIVE)$"
            }
        },

        {
            "_comment": "EXAMPLE 3: Exclude all
objects from specific schemas",
```

```
            "name": "exclude_test_schemas",
            "description": "Exclude all objects
from test/dev/qa schemas",
            "action": "EXCLUDE",
            "match": {
                "owner": "^(TEST|DEV|QA)_.*"
            }
        },

        {
            "_comment": "EXAMPLE 4: Exclude
specific objects by exact match (no regex)",
            "name": "exclude_specific_objects",
            "description": "Exclude specific known
objects that should be ignored",
            "action": "EXCLUDE",
            "objects": [
                {
                    "_comment": "Explicit object
1",
                    "owner": "HR",
                    "name": "EMPLOYEES_AUDIT",
                    "type": "TABLE"
                },
                {
                    "_comment": "Explicit object
2",
                    "owner": "SCOTT",
                    "name": "LEGACY_PROC",
                    "type": "PROCEDURE"
                }
            ]
        },

        {
            "_comment": "EXAMPLE 5: Apply rule only
to specific checks",
            "name": "exclude_java_from_java_check",
            "description": "Exclude Java classes
only from the CHECK_JAVA_OBJECTS check",
            "action": "EXCLUDE",
            "check_names": ["CHECK_JAVA_OBJECTS"],
            "match": {
                "object_type": "JAVA CLASS"
            }
        },

        {
            "_comment": "EXAMPLE 6: Exclude
multiple object types at once",
            "name": "exclude_legacy_code",
            "description": "Exclude stored
procedures, functions and packages from
LEGACY_SCHEMA",
            "action": "EXCLUDE",
            "match": {
                "owner": "^LEGACY_SCHEMA$",
                "object_type": ["PROCEDURE",
"FUNCTION", "PACKAGE", "PACKAGE BODY"]
            }
        },
```

```
        {
            "_comment": "EXAMPLE 7: INCLUDE rule
(whitelist mode) - when present, only matching
objects are
included",
            "name":
"include_production_schemas_only",
            "description": "Only include objects
from production schemas (use with caution - creates
whitelist)",
            "action": "INCLUDE",
            "match": {
                "owner": "^(PROD|APP|CORE)_.*"
            }
        }
    ],

    "_rule_reference": {
        "name": "Unique identifier for the rule
(used in filter metadata/logs)",
        "description": "Human-readable description
of what the rule does",
        "action": "INCLUDE or EXCLUDE - determines
what happens when rule matches",
        "match": {
            "owner": "Regex pattern to match object
owner/schema",
            "object_name": "Regex pattern to match
object name",
            "object_type": "Single type string or
array of types (TABLE, PROCEDURE, FUNCTION, VIEW,
etc.)"
        },
        "objects": "Alternative to 'match' - list
of explicit objects with exact owner/name/type",
        "check_names": "Optional - apply rule only
to specific CPAT checks (array of check names)"
    }
}
```

The filter file and the other filter parameters can be used contiguously as needed.

Checking for a Updated Version of CPAT

Please check the version of CPAT installed by running CPAT with the "**--version**" switch.

Please check for an updated version of CPAT by running CPAT with the "**--updatecheck**" switch.

Note: Successful use of "--updatecheck" requires that CPAT be run from an environment with internet (HTTPS) access. If your environment is behind a firewall the environment must also contain properly defined proxy settings.

CPAT Integration with Other Oracle Products

CPAT has been integrated with

- Oracle Zero Downtime Migration
- Oracle Database Migration Service
- Oracle Enterprise Manager Migration Workbench
- Oracle SQLcl, SQL command line

Updating CPAT in a Zero Downtime Migration Home

Note: Generally you should only update ZDM HOME with newer versions of CPAT. If you need to revert to an earlier version of CPAT then the version of CPAT you're moving to cannot be older than the version of CPAT that originally shipped in your ZDM distribution. Step two below includes the command to find that version number.

Note: Avoid starting migration jobs during the update process.

CPAT is bundled with ZDM version 21c and higher

To install a newer version of CPAT in the ZDM Home the following process should be used:

1. Download the new version of CPAT (see the download instructions above)
2. Run "./ZDM_home/rhp/zdm/lib/cpat/premigration.sh

--version" to see what is currently installed in your ZDM home
3. Backup the currently installed CPAT by making a copy of ZDM_home/rhp/zdm/lib/cpat
4. Delete all files within ZDM_home/rhp/zdm/lib/cpat
5. Unzip the new CPAT release file to ZDM_home/rhp/zdm/lib/cpat
6. If needed, change the permissions on the unzipped files to remove unwarranted access.
7. Run "./ZDM_home/rhp/zdm/lib/cpat/premigration.sh --version" to verify the newer version is in place

Note: It is not necessary to stop and restart the zdmservice after updating CPAT.

Migration jobs can be submitted after updating CPAT.

Using CPAT with Other Tooling or Scripted Processes

CPAT was designed with integration into other tools and products in mind.

For example, a primary output format for CPAT reports is JSON. This allows integrators to pull whatever subset of information is appropriate for their needs from the CPAT report, format that information as they see fit, and discard the rest.

A simpler integration with CPAT could rely on CPAT's shell script exit code and TEXT or HTML report formats. The CPAT shell script will return values that reflect its ability to run and analyze the given instance. This allows any script calling CPAT to do basic conditional processing without having to parse the CPAT JSON report.

CPAT Shell Script Exit Codes

The following table lists each of the possible exit codes returned by the CPAT shell script and under what conditions those exit codes are returned.

Shell Script Exit Codes for CPAT

Exit Code	Conditions under which that exit code is returned
254	If the script determines the current environment prevents Java from running correctly it will return 254. For example, 254 is returned if ORACLE_HOME and/or JAVA_HOME isn't defined or the JVM isn't Java 7 or higher.
255	If nothing prevents Java from running but an irrecoverable error error occurs before or during Java processing then 255 is returned. This could happen if CPAT is unable to obtain a JDBC connection to the instance. For example the connect string was improperly specified or the supplied credentials were incorrect.
3	If CPAT analysis completes and the generated report indicates at least one of the checks had an **Action Required** result then 3 is returned.
2	If there were no **Action Required** results but CPAT analysis indicates at least one of the checks had a **Review Required** result then 2 is returned.
1	If there were no **Action Required** or **Review Required** results but analysis indicates at least one of the checks had a **Review Suggested** result then 1 is returned.
0	If CPAT analysis completes and the generated report indicates that ALL checks had a **Passing** result then 0 is returned.

Security Notes

The Reports generated by CPAT may contain sensitive information. Therefore, when running CPAT, one should take care to use the '--outdir' switch to set the output location of our reports/logs to a secure location.

Similarly, on Unix platforms, CPAT users should also set their umask appropriately.

Summary

The Cloud Premigration Advisor Tool (CPAT) can provide insight and guidance for potential issues migrating to an Oracle Cloud Database.

Use of CPAT does not require modifying the source or target instance.

RECENT_CHANGES.pdf
cpat_cli_help_output.pdf

Product Versions

Product: Autonomous Database Dedicated - min_version: NA - max_version: none
product: Autonomous Database Serverless - min_version: N/A - max_version: none

Internal Only

The CPAT slack channel, #cpat-premigration-advisor, should be used for any questions.

Article Feedback

☆☆☆☆☆ Rate this

© 2026 Oracle | Terms of Use | Privacy